

Asymptotic

Componentwise validation
and standalone parsing

Source review for Wolfram Language 15 and Mathics3

Repository	VladimirReshetnikov/Asymptotic
Pinned revision	efa1aeec4845a9c35e140963a0333d0c9ec33b05
Commit time	10 September 2026, 19:09:04 UTC
Exclusion baseline	Six review-wave indexes and the maintained status/intake summaries, with targeted comparison to nearby original reports
New findings	One component-validation defect with a mathematically false remainder counterfamily; one latent Mathics bootstrap splitting defect
Executed evidence	33 independent Python test methods; no successful Wolfram or Mathics package execution

Prepared for Vladimir Reshetnikov

10 September 2026

Evidence boundary. The mathematical calculations and Python helper experiments in this report were executed independently. The proposed public Wolfram calls, the supplied MUnit regressions, the Mathics behavior, and application of the patch to a real checkout remain unexecuted. A source prediction is never counted as a runtime result.

Executive assessment

This audit isolates two specific mechanisms beyond the inspected prior-review inventory. It deliberately does not reissue the existing numerical-certificate, assumption-capture, remainder-calculus, option-handling, sparse-arithmetic, or general portability backlog. The principal correctness issue is not an exotic special-function identity: a constructor validates an algebraically simplified *sum* while its subsequent theorem concerns the two summands *separately*. The principal distribution issue is similarly structural: a source splitter recognizes ordinary brackets but not association delimiters.

N01 — component dependence can disappear before validation. In `AsymptoticCoreInverse`, target-dependent offsets in the core and perturbation can cancel in `core+perturbation`. The check therefore admits the combined expression, although the separately parsed components no longer satisfy the fixed-parameter hypothesis used to assign a remainder. For the source-predicted call

```
AsymptoticCoreInverse[
  x + Abs[y]/2, -Abs[y]/2, {x, Infinity}, {y, 1},
  "Power" -> 2]
```

the inspected formulas give, on the positive target ray,

$$\text{finite approximation} = \frac{3}{4}y^2, \quad \text{claimed remainder scale} = 1.$$

The total forward function is exactly x , so the requested inverse observable is y^2 . Its error is $y^2/4$, not $O(1)$. The article traces the intervening real-coefficient checks, gives a smaller depth-zero example, and proves a family of failures. This is a high-priority validation/correctness defect for inputs the constructor intends to reject, not a claim that valid fixed-parameter inputs fail. The repair is to validate `{core, perturbation}` and test that pair for target dependence.[2, 3]

N02 — association syntax is not counted as nesting in the standalone bootstrap. The Python builder splits a `Mathics` adapter into strings that will be separately parsed by `ToExpression`. Its nesting counter ignores `<|` and `|>`. A semicolon inside an association therefore splits one expression into unmatched fragments. Local execution of the transcribed helper demonstrates three chunks where two top-level statements are required. The supplied delimiter-stack candidate repairs that case and preserves the previously handled condition and span operators. No present shipped adapter triggering this syntax is claimed; this is a latent generator/portability defect, not an observed failure of the current distribution to load.[5, 6, 22]

All **33 independent test methods pass**, with zero failures, errors, or skips. They exercise exact mathematical identities, a manually transcribed two-function builder fixture, the candidate splitter, and synthetic patch anchors. They are not 33 package acceptance tests. The runtime services and local installation routes did not yield a functioning `Wolfram` or `Mathics` evaluator during this audit.

Contents

Executive assessment	1
1 Scope, baseline, and what was actually established	3
1.1 Snapshot and review method	3
1.2 Evidence classes	3
1.3 Non-duplication boundary	3
2 The relevant design contracts	4
2.1 A positive source coordinate and a fixed family	4
2.2 Two expression representations that must not be confused	4
3 N01: cancellation bypasses componentwise target validation	5
3.1 Affected site and severity	5
3.2 Why the coefficient-reality check matters	5
3.3 Smallest counterexample: the inverse itself at depth zero	6
3.4 A depth-one example with a squared observable	6
3.5 General counterfamily and exact proof	7
3.6 Even a relatively small moving offset is enough	7
3.7 Narrow repair	8
3.8 Controls and acceptance criteria	8
4 N02: association delimiters are missing from the Mathics splitter	9
4.1 Why the builder splits source	9
4.2 The concrete fixture and observed chunks	9
4.3 Language support for the fixture	9
4.4 A metamorphic test exposes the omission	10
4.5 Impact and limits	10
4.6 Candidate implementation and complexity	10
4.7 Acceptance tests and integration path	11
5 Wolfram 15 and Mathics3: the justified compatibility assessment	11
6 Performance, API, and development consequences	12
6.1 No additional performance defect is promoted	12
6.2 Make the component contract explicit, not merely the total-function contract	12
6.3 A bounded mathematical extension, not a silent relaxation	13
6.4 Preserve syntax under equivalent grouping spellings	13
7 Repair order and verification gates	14
8 Reproducibility and artifact inventory	14
8.1 Independent execution	14
8.2 Candidate source change	14
8.3 Unexecuted runtime probes	15
8.4 Contents and build	15
9 Conclusion	15
A Inspected source areas and exclusions	16
B Source locators and bibliography	17

1 Scope, baseline, and what was actually established

1.1 Snapshot and review method

The review pins revision `efa1aeec4845a9c35e140963a0333d0c9ec33b05`. The authoritative code examined is the modular source under `src/Kernel`; the standalone builder and its tests were read separately. The snapshot is important because the repository’s retained review collection is changing rapidly. The wave-6 index at this revision contains reports 46–55 and records that its underlying source mechanisms are unchanged from the wave’s common earlier source baseline.[1, 16]

The inspection followed public construction, coefficient normalization, marker inversion, generalized-series arithmetic, transformed coordinates, native special-function import, Mathics adaptations, and selected validation/build tooling. It compared candidate findings with the six wave indexes, the maintained status register, and their intake/proposal summaries. Nearby original material was read where a distinction mattered, particularly report 43’s formula-generator validation issue and report 46’s exponential source-shift issue.[10, 9, 11, 12, 13, 14, 15, 16, 17, 18]

A full repository clone was not obtained. Commit-pinned GitHub connector reads supplied the source; local download and installation routes failed. Consequently, this is a substantial *source audit with independent experiments*, not a checkout-based acceptance run, a package benchmark, or a proof of interpreter equivalence. Appendix A lists the inspected areas without implying exhaustive line-by-line coverage of every file.

1.2 Evidence classes

The following distinctions are used throughout.

Class	Meaning in this report
Source-established	A control-flow or data-flow property of definitions actually read at the pinned revision.
Mathematically established	An exact derivation from stated formulas and assumptions; independently checked where indicated.
Python-observed	An executed experiment in the supplied Python model, fixture, or candidate code.
Source-predicted public result	A proposed package outcome derived from source, with no successful interpreter execution of that call.
Unexecuted integration test	A supplied WL/WLT test or probe that specifies what should be checked in an actual runtime.

For N01, the validation defect and mathematical contradiction are established at the first two levels. The public `Abs[y]` witness remains a source prediction. For N02, incorrect chunk boundaries are Python-observed in the transcribed helper; the parse consequence is supported by the language grammar. Neither observation establishes a current package load failure.

1.3 Non-duplication boundary

Novelty here means a distinct mechanism and required repair not found in the inspected inventory. It does not mean that every sentence of every archived article was compared. Broad principles such as “parameters must remain fixed” and “standalone loading must preserve syntax” are not claimed as new.

Finding	Nearest existing material	Distinct obligation here
N01	Report 43 F03; reports 46 N01 and 50 F1	A sum/pair mismatch in CorePerturbation.w1 . The witness uses no SourceShift option and no user-supplied inverse expression. Neither previous guard repair changes this site.
N02	D04 and prior standalone/bootstrap hardening	The present splitter’s missing association delimiter family, with a concrete internal-semicolon fixture. Existing condition/span tests do not exercise it.

Report 43’s F03 concerns a purported inverse supplied to **PerturbativeInverse** that retains the source symbol being eliminated. Reports 46 and 50 concern a target-dependent **SourceShift** admitted by **AsymptoticExponentialCoreInverse**. They are adjacent fixed-parameter validation problems, but their affected arguments, constructors, and replacement lines differ from N01. The present repair preserves that distinction instead of expanding the old reports into a generic repeated recommendation.[17, 18, 16]

For N02, the existing builder already masks strings and nested comments, delays Mathics parsing, and handles the semicolons embedded in /; and ;;. Those protections are retained, not presented as absent. The new failure is specifically association nesting.[5, 6]

2 The relevant design contracts

2.1 A positive source coordinate and a fixed family

At a positive infinite source endpoint, the package uses

$$u = \frac{1}{x}, \quad u \longrightarrow 0^+.$$

The exact-core constructor separates a core F_0 from a perturbation H , chooses an exact inverse of the core, and forms coefficients in a marker λ for

$$F_0(x) + \lambda H(x) = y. \tag{1}$$

The marker is set to one after truncation. The source model records the leading power p , the minimum perturbation gap $\delta > 0$, logarithmic information, and the local power r_{loc} of the requested observable. At infinity, a positive integer observable power r gives $r_{\text{loc}} = -r$. [2]

The advertised marker-tail bound is an existence statement for *fixed data sufficiently close to the endpoint*. Its constants and source threshold are not computed numerical bounds. The recorded truncation power is

$$\rho = r_{\text{loc}} + (n + 1)\delta, \tag{2}$$

with a separately computed logarithmic degree. Equation (2) is meaningful only if the quantities treated as coefficients really are fixed on the target approach, or a uniform theorem has replaced that premise. [2]

2.2 Two expression representations that must not be confused

The combined forward function $F_0 + H$ and the decomposition (F_0, H) serve different roles. Equality of two combined expressions says nothing by itself about whether their decompositions obey the same

hypotheses. In particular,

$$(x + A(y)) + (-A(y)) = x$$

removes all visible target dependence from the sum but not from the two inputs supplied to the exact-core algorithm. Replacing the pair by its sum is an information-losing transformation *before* checking a componentwise condition.

There is a parallel distinction in the builder. A complete adapter source and a list of separately parsed source strings are not interchangeable unless chunk boundaries respect every supported grouping form. Ordinary bracket balance is not enough when the language also has association delimiters. These two boundaries motivate the findings below; they do not imply that the whole repository should be redesigned around a new representation.

3 N01: cancellation bypasses componentwise target validation

3.1 Affected site and severity

The affected entry is `corePerturbationConstruct` in `CorePerturbation.wl`, lines 113–153. Its current initial checks include:

```
validateInput[core + perturbation, limit];
If[x === y || ! FreeQ[core + perturbation, y],
  fail["InvalidVariables", ...]];
```

The public `AsymptoticCoreInverse` wrapper forwards the core and perturbation as separate arguments. Later code separately substitutes the positive local coordinate into each, parses their coefficient rows, and assigns the gap-based remainder. There is no corresponding componentwise target-independence test at that parsing boundary.[2]

Severity: high correctness impact, restricted to invalid decompositions that should be rejected. The resulting failure can be a successful-looking finite expansion with an impossible asymptotic bound. It is not merely a less sharp error estimate, an expensive fallback, or a missing convenience feature. The source-predicted public result still requires runtime confirmation, as discussed below.

3.2 Why the coefficient-reality check matters

A naive witness with $x+y/2$ and $-y/2$ is insufficient by itself. The model checks the realness of coefficient polynomials, and an unconstrained symbolic y need not pass that proof. Adding `Assumptions->y>0` is not a legitimate workaround: this constructor separately requires assumptions to concern parameters rather than source or target variables.[2, 3]

The principal witness therefore uses $A(y) = |y|/2$. This is a real-valued coefficient expression without putting the target in the constructor’s assumptions. On the positive real target ray, it becomes $y/2$. The ordinary Wolfram realness simplifier is expected to recognize this coefficient; the probe explicitly records that subquery before the public calls. This expected behavior was not executed here.

The Mathics adapter has a narrower structural realness prover: its `Abs` case recursively asks for realness of the inner expression. Thus this particular witness may be rejected conservatively by Mathics before the wrong remainder is constructed. That possibility is retained in the evidence record. It does not repair the shared sum/pair validation site, and it is not presented as another new Mathics defect.[7]

3.3 Smallest counterexample: the inverse itself at depth zero

Consider the proposed public call

```
s = AsymptoticCoreInverse[
  x + Abs[y]/2, -Abs[y]/2,
  {x, Infinity}, {y, 0}];
```

Assuming the real-coefficient checks accept the stated real-valued coefficient, the inspected source gives the following path.

The combined forward expression simplifies to x , so the initial target check sees no y . Separately, the local model is

$$F_0(u; y) = u^{-1} + \frac{|y|}{2}, \quad H(u; y) = -\frac{|y|}{2}.$$

The leading core power is $p = -1$, its amplitude is one, the offset is $|y|/2$, and the perturbation has power zero. Hence $\delta = 1$, with logarithmic degree zero. The automatic core inverse and its reciprocal coordinate are

$$\phi(y) = y - \frac{|y|}{2}, \quad u_0(y) = \frac{1}{y - |y|/2}.$$

On the positive ray these become $y/2$ and $2/y$. Depth zero retains only ϕ . For $r = 1$, equation (2) gives

$$\rho = -1 + (0 + 1) \cdot 1 = 0.$$

The source consequently assigns `PowerLogRemainder[u0, 0, 0]` and scale one.[2]

The default source-radius restriction is compatible with this ray. On positive real y , the recorded requirement $0 < u_0 < 1/e$ holds for $y > 2e$; the other displayed sign conditions also hold there. The problem cannot be dismissed as a contradictory domain.

But the total forward function is exactly x , whose inverse is exactly y . The actual error is

$$y - \frac{y}{2} = \frac{y}{2} \longrightarrow +\infty.$$

Therefore the asserted $O(1)$ remainder is false. A larger hidden constant cannot repair it.

3.4 A depth-one example with a squared observable

The phenomenon is not confined to a zero-depth request. Take

```
s2 = AsymptoticCoreInverse[
  x + Abs[y]/2, -Abs[y]/2,
  {x, Infinity}, {y, 1}, "Power" -> 2];
```

For a fixed target value during marker differentiation,

$$x_\lambda = \left(\frac{1}{2} + \frac{\lambda}{2}\right)y, \quad x_\lambda^2 = \frac{y^2}{4} + \lambda \frac{y^2}{2} + \lambda^2 \frac{y^2}{4}.$$

Depth one gives $3y^2/4$. Here $r_{\text{loc}} = -2$ and $\delta = 1$, so $\rho = -2 + 2 = 0$. The envelope is again one, while the omitted error is exactly $y^2/4$.

For $r = 1$, increasing the marker depth to one happens to make the finite expression equal to the exact inverse. That is a property of this affine marker polynomial; it does not validate the guard. The constructor may still record a conservative nonzero remainder. Tests that only compare finite expressions at a convenient depth can therefore miss the malformed-input defect entirely.

3.5 General counterfamily and exact proof

Let $0 < c < 1$, write $b = 1 - c$, and use

$$F_0(x; y) = x + c|y|, \quad H(x; y) = -c|y|.$$

The total forward function is still x . Restrict to $y > 0$, as allowed by the retained target approach.

Proposition 3.1. *For an integer observable power $r \geq 1$ and depth $0 \leq n < r$, the inspected marker construction predicts*

$$P_n(y) = y^r \sum_{k=0}^n \binom{r}{k} b^{r-k} c^k, \quad (3)$$

$$R_n(y) = (by)^{r-n-1}. \quad (4)$$

Its exact observable error is a positive constant times y^r , and

$$\frac{y^r - P_n(y)}{R_n(y)} = b^{n+1-r} \left(\sum_{k=n+1}^r \binom{r}{k} b^{r-k} c^k \right) y^{n+1} \rightarrow +\infty. \quad (5)$$

Proof. Equation (1) becomes $x_\lambda = (b + c\lambda)y$. Expanding its r -th power gives (3). The local core inverse is $u_0 = (by)^{-1}$, and the stored model has $\delta = 1$ and zero logarithmic degree. Thus (2) gives $\rho = -r + n + 1$ and $u_0^\rho = R_n$. The full binomial sum is $(b + c)^r = 1$. Every omitted term is positive because $b, c > 0$, so subtraction and division give (5). $\square$

The supplied symbolic check also verifies the actual displayed differentiation formula used for marker coefficients, rather than checking only the binomial identity. Holding A fixed during differentiation, let $F_0 = u^{-1} + A$, $H = -A$, and the observable be u^{-r} . The formula in `corePerturbationTerm` yields

$$\frac{(-1)^k}{k!} \left(\frac{1}{F_0'(u)} \frac{d}{du} \right)^{k-1} \frac{(u^{-r})' H(u)^k}{F_0'(u)} = \binom{r}{k} A^k u^{k-r}.$$

The independent SymPy test checks this identity for $r = 1, \dots, 6$, including the first coefficient after polynomial termination. This verifies the narrow formula specialization, not the full package dispatcher.

3.6 Even a relatively small moving offset is enough

One might attempt to repair the problem merely by checking that the perturbation is small relative to the core. That is not sufficient to justify the *existing* pure-power remainder scale.

Let $A(y) = \log(1 + y)$ on the positive ray, represented in an input by `Log[1+Abs[y]]`. Then $A(y)/y \rightarrow 0$, but $A(y) \rightarrow \infty$. At $r = 1, n = 0$, the predicted finite expression is $y - A(y)$, the actual error is $A(y)$, and the assigned scale remains one. The error is still not $O(1)$.

Proposition 3.2. *Suppose $A(y) \geq 0$, $A(y) = o(y)$, and $A(y) \rightarrow \infty$. For fixed integers $r \geq 1$ and $0 \leq n < r$, define the marker truncation*

$$P_n(y) = \sum_{k=0}^n \binom{r}{k} (y - A(y))^{r-k} A(y)^k.$$

Against the scale predicted by the fixed-coefficient model, $R_n(y) = (y - A(y))^{r-n-1}$, one has

$$\frac{y^r - P_n(y)}{R_n(y)} \sim \binom{r}{n+1} A(y)^{n+1} \rightarrow +\infty.$$

Proof. Subtract the truncated binomial sum from the complete one and divide by R_n . The exact normalized error is

$$\sum_{k=n+1}^r \binom{r}{k} A(y)^k (y - A(y))^{n+1-k}.$$

Factoring $A(y)^{n+1}$ leaves a polynomial in $A(y)/(y - A(y))$ whose constant term is $\binom{r}{n+1}$. That ratio tends to zero. Positivity excludes cancellation. $\square$

This identifies the precise missing premise: coefficient constants cannot grow with the target unnoticed. In the linear counterfamily even relative smallness fails; in the logarithmic family relative smallness holds, but the uniform scale is still wrong. The latter is a stronger reason to repair the input contract rather than add one ad hoc asymptotic sign or ratio test.

3.7 Narrow repair

The candidate change is:

```
validateInput[{core, perturbation}, limit];
If[x === y || ! FreeQ[{core, perturbation}, y],
  fail["InvalidVariables",
    "Use distinct source and target symbols; the core and perturbation must each be
    independent of the target symbol."]];
```

The combined expression may still be used later for **Function** metadata. The change only prevents it from replacing the pair when checking a condition that must hold of both components.

Using the pair also preserves the existing exact-input screen across cancellation between inputs. For example, two separately inexact constant offsets should not become admissible merely because their sum disappears. This is one consequence of the same validation repair, not a separately counted defect. The change does not attempt to recover syntax already erased by ordinary evaluation *within* a single argument, nor does it require making the whole public API held.

The supplied patch emitter requires exactly one old anchor, rejects an already-patched or ambiguous input, and prints a diff without modifying the source. Its tests use synthetic anchors, not a real checkout. The candidate patch must be reviewed and applied to the canonical module, and the standalone must then be rebuilt through the repository's existing mechanism.

3.8 Controls and acceptance criteria

A satisfactory repair rejects both the depth-zero inverse witness and the powered depth-one witness with **InvalidVariables**. It must also reject a renamed target and a real-part spelling of target dependence, so the guard is not tailored to **Abs**.

Conversely, it must continue to admit genuinely fixed data. For $F_0 = x + a$, $H = -a$, with fixed real a , the depth-zero error is exactly $a = O(1)$. For the squared observable at depth one the error is $a^2 = O(1)$. These are valid controls, not near misses. The supplied WLT tests also include the ordinary fixed perturbation $H(x) = 1/x$, preservation of the distinct-source/target check, and component exactness checks.

Twelve desired-contract MUnit tests and a portable-style observation probe are included. They were not executed. The probe prints the result head before accessing series-specific fields and records

the target domain, remainder, envelope, and the exact error on the positive ray. It also prints the coefficient-reality subquery and the naive plain-target control, making an interpreter-specific earlier rejection visible rather than silently assuming reachability.

4 N02: association delimiters are missing from the Mathics splitter

4.1 Why the builder splits source

The standalone builder in `validation/build_standalone.py` expands modular companion loads. For Mathics-only adapters, it delays parsing as well as evaluation on the Wolfram path by serializing top-level statements and evaluating them through `Scan[ToExpression, {...}]`. This preserves streaming context changes while avoiding premature binding of Mathics-specific definitions in the native kernel.^[5]

The `mathics_bootstrap` helper obtains a masked source from `executable_text`. It then increments nesting depth for `[{ (, decrements it for }], and regards a semicolon at depth zero as a statement boundary. It explicitly skips the two-character operators /; and ;;. Association delimiters are absent from this nesting logic.[5]`

4.2 The concrete fixture and observed chunks

Use the source

```
settings = <| counter = 1; "key" -> counter |>;
after = 2;
```

The interior semicolon belongs to a compound expression inside the association. The reference helper transcribed from the inspected file produced these three strings in the executed Python experiment:

```
settings = <| counter = 1;
"key" -> counter |>;

after = 2;
```

The first and second strings are separate chunks; the blank line above the third reflects its leading newline, not an additional chunk. The first has an unmatched opening association delimiter and the second has an unmatched closing delimiter. Parsing them independently cannot reproduce parsing of the complete association expression.

The reference's counter remains zero after `<|`, so the first internal semicolon is inevitably treated as top level. The fixed candidate returns two chunks, keeping the association intact. The exact input and both chunk lists are retained in `evidence/bootstrap-fragments.json`.

This experiment executes a *manual transcription of two helper functions*. It is not a run of the complete repository builder. The transcription provenance is explicit in the fixture and notice; no byte-identity or successful upstream build is claimed.

4.3 Language support for the fixture

Association notation is a language grouping form, not a comment-like visual abbreviation. Wolfram documents its equivalence to `Association[...]` and identifies the named association characters as a

matched operator pair. Its documented character codes are U+F113 and U+F114.[19, 20, 21]

The inspected primary Mathics parser independently confirms the relevant nesting rule. Its `p_LessBar` routine increments `bracket_depth`, parses an argument sequence, expects `BarGreater`, and only then decrements the depth. The sequence parser accepts general expressions, and its semicolon handler constructs compound expressions within bracketed syntax. This is grammar evidence, not an execution of that parser.[22]

The report makes no claim about the ordinary evaluated value of every association fixture. The contract being tested is preservation of a parseable source expression across chunking; changing one complete grouped expression into separately unparsable fragments is already a generator defect. The supplied WL syntax probe requests held parses and leaves evaluation parity as a separate integration check.

4.4 A metamorphic test exposes the omission

Replace the association’s delimiter spelling by an explicit constructor:

```
settings = Association[counter = 1; "key" -> counter];
after = 2;
```

The reference splitter now returns two chunks because it counts the ordinary square bracket. The desired chunk-boundary property is invariant under this syntactic spelling change, yet the current implementation violates it. This is a useful regression because it does not depend on any special-function expansion, timing threshold, or numerical oracle.

The existing builder tests already exercise string/comment masking, unbalanced ordinary brackets, streaming context statements, and condition/span operators. Their bootstrap fixtures do not contain the association/internal-semicolon combination. The new tests extend this particular syntax population; they do not replace those existing controls.[6]

4.5 Impact and limits

Severity: medium, latent distribution/portability defect. A future legal adapter statement using this top-level association form can work as a complete modular source expression while the generated standalone splits it incorrectly. The generated failure would affect the Mathics-only bootstrap path. The Wolfram path does not evaluate that conditional bootstrap.

No current shipped adapter is identified as using the triggering form. No current standalone load failure was reproduced in either interpreter. Accordingly, this finding must not be summarized as “the package cannot load on Mathics.” It is a concrete flaw in the builder’s advertised source-handling logic and an avoidable maintenance hazard.

The depth-only implementation also fails to distinguish mismatched ordinary delimiter kinds. For example, `f[x]`; has net depth zero despite mismatched brackets. The candidate rejects that malformed input while repairing association handling. This is treated as a supporting delimiter-invariant test, not another numbered finding or a claim that the repository presently ships malformed source.

4.6 Candidate implementation and complexity

The supplied `bootstrap_fixed.py` uses a stack of expected closing-token families. It recognizes ordinary parentheses, brackets, braces, and association delimiters in ASCII, long-name, and documented

character form. Masked strings and comments do not affect the stack. The two semicolon-containing operators retain their prior special treatment.

A semicolon is a boundary only when the stack is empty. A closing token must match the expected family; an unclosed group or mismatched closer raises an error with an offset. This makes the preprocessing failure earlier and more intelligible without relying on the generated runtime to report a syntax error.

For source length L and maximum nesting depth D , the fixed token vocabulary is constant. Masking, scanning, slicing all emitted chunks, and serialization take $O(L)$ work and $O(L)$ total storage; the grouping stack needs $O(D)$ additional space. This is a code-derived complexity statement, not an elapsed-time benchmark of the package. The correction does not introduce symbolic simplification or numerical work.

The candidate is *not a full Wolfram parser*. It retains the reference builder’s semicolon-terminated statement convention. Box syntax and explicitly recognized unsupported grouping spellings are refused; other language-wide parsing features have not been certified. It is suitable as a reviewable narrow prototype, not evidence that arbitrary Wolfram source can safely be split without the language parser.

4.7 Acceptance tests and integration path

The independent tests verify the observed three-chunk defect, the corrected two-chunk result, constructor-spelling invariance, three association delimiter spellings, nested associations, strings/comments, condition/span preservation, mismatched and unclosed groups, serialization round trips, and a small generated balanced-grammar grid.

The integration sequence should keep the two obligations separate: first show that the whole fixture has a held parse while each broken reference fragment does not; then compare modular loading with the rebuilt standalone in a fresh Mathics process. Native Wolfram loading remains a control for the nonexecuted Mathics branch. The included syntax probe addresses the first obligation but has not been run. The candidate module has not been transplanted into a real checkout, and the repository’s standalone build has not been executed here.

5 Wolfram 15 and Mathics3: the justified compatibility assessment

The source contains a deliberate dual-runtime architecture. The main loader conditionally loads Mathics-specific adapters before the relevant analytic definitions, and the standalone builder defers parsing of those adapters. A review must therefore distinguish common mathematical code, adapter behavior, and generated-distribution behavior rather than treat one interpreter’s successful evaluation as proof for the other.[4, 5, 7, 8]

Question	Wolfram Language 15	Mathics3
N01 guard location	Shared constructor contains the sum/pair mismatch.	Same shared constructor; an earlier adapter-specific proof failure may mask the witness.
N01 public witness	Source-predicted; real-valued <code>Abs[y]</code> coefficient chosen to address the reality guard.	Unexecuted; the narrower structural realness prover may refuse it.
N02 preprocessing	Python helper defect exists independently of the native interpreter.	Wrong chunks would be parsed in the Mathics-only bootstrap path.
Package acceptance	Not run.	Not run.
Patched distribution	Not rebuilt or loaded.	Not rebuilt or loaded.

The independent Python results cannot be used to certify compatibility with either runtime. In particular, the Mathics parser source establishes syntax rules, not behavior of the package’s installed adapter stack. The runtime probes record version and actual result heads precisely to prevent this distinction from being lost when the tests are subsequently executed.

N01’s repair is deliberately interpreter-independent: reject structurally invalid component dependence before asking either engine to prove coefficient realness. A conservative Mathics refusal for a different reason is not an adequate long-term replacement for the public input contract. N02’s repair similarly belongs in the distribution builder, not in a runtime workaround that catches malformed generated chunks after the fact.

6 Performance, API, and development consequences

6.1 No additional performance defect is promoted

Sparse products, coefficient normalization, replay paths, flat-sector products, native export boundaries, and Mathics helper recursion were inspected. Several apparent opportunities belonged to already-recorded review obligations; they are not relisted here as new findings. This audit does not claim a fresh upstream timing regression or supply a package benchmark.

For the two retained repairs, the performance consequences are bounded and direct. Componentwise structural screening should occur before exact-core parsing, automatic inverse construction, and marker differentiation, avoiding work on a request known to violate the API. The delimiter-stack repair adds only linear source scanning and linear output storage. These statements describe the proposed fixes, not measured speedups.

6.2 Make the component contract explicit, not merely the total-function contract

N01 calls for one narrow API clarification: the core and perturbation must *each* be independent of the target coordinate. “The forward expression must not contain the target” is ambiguous when the public API accepts a decomposition and later recomposes it. The proposed diagnostic names both components and avoids changing existing option semantics.

The associated regression relation is also specific. Adding $+A(y)$ to the core and $-A(y)$ to the perturbation preserves the total expression but must change the validity classification when A contains

the target. Adding genuinely fixed $+a, -a$ must not do so. This is a contract-sensitive transformation test, not a demand that every algebraically equal decomposition return the same finite marker expansion.

6.3 A bounded mathematical extension, not a silent relaxation

Supporting deliberately target-dependent decompositions would require a different public contract. Proposition 3.2 shows why proving only $H/F_0 \rightarrow 0$ cannot justify the old error fields. The coefficient envelope itself must be transported.

For the simple affine family, a future explicitly uniform constructor could retain

$$\sum_{k=n+1}^r \binom{r}{k} |A(y)|^k |y - A(y)|^{r-k}$$

as a finite exact tail envelope for integer observables. For nonnegative bounded $A(y)$, the existing scale can be recovered with a constant depending on a proved bound for A . For unbounded A , its growth must remain in the envelope. This is a small, proved extension target arising from N01; it is not a recommendation to admit such inputs into the current fixed-data API.

A reasonable decision is therefore to implement the rejection first. An eventual uniform-family feature should have a separate entry point or explicit contract, identify which coefficients may vary, and supply a theorem for its envelope. Renaming metadata or weakening a diagnostic without that theorem would repeat the mathematical mistake.

6.4 Preserve syntax under equivalent grouping spellings

N02 suggests a similarly bounded development direction: the standalone builder's supported-source contract should include a syntax-spelling matrix for grouping forms. Its fixtures should compare `Association[...]` with association delimiters and retain semicolon, comment, and string variants within those groups. This is a targeted extension of the current builder tests, not a general CI redesign.

If the supported adapter language eventually expands beyond a small, semicolon-terminated source subset, the decision should be explicit: either delegate statement boundaries to a Wolfram-compatible parser, or refuse unrecognized grouping syntax with a build-time diagnostic. The included prototype chooses a limited stack-based subset and documents what it does not establish.

7 Repair order and verification gates

Stage	Change	Evidence required before closing the finding
1	Replace the two N01 sum-based checks by pair-based checks.	Public malformed-input rejections and fixed-parameter controls in Wolfram 15; repeat on Mathics, recording any earlier interpreter-specific result.
2	Add association nesting to bootstrap splitting without regressing <code>/;</code> or <code>;;</code> .	Run the actual repository builder tests plus the new syntax fixtures; compare held parse outcomes.
3	Rebuild the standalone from canonical sources.	Run modular and standalone load/control probes independently in fresh processes.
4	Consider the uniform affine-family extension only as a separate feature.	A documented domain and coefficient-dependent envelope, with an independent exact oracle.

The current archive supplies the narrow candidate code, independent tests, and unexecuted integration specifications for the first three stages. It does not report those stages as completed in the repository. No repair is silently applied to the user’s GitHub data.

8 Reproducibility and artifact inventory

8.1 Independent execution

Run from the archive root:

```
python code/run_independent.py
```

The run requires Python and SymPy. The recorded environment is Python 3.13.5 with SymPy 1.14.0. The test population is 13 mathematical-model methods, 15 bootstrap/lexer methods, and five patch-emitter methods: 33 in total. Parameter grids and subcases are not added to that method count. The JSON receipt and text transcript are in **evidence/**.

The mathematical tests establish pair-versus-sum dependency loss in an independent expression system, the depth-zero and squared examples, marker-coefficient identities, positivity of omitted tails, exact growth ratios, polynomial termination controls, fixed-offset controls, the sublinear-offset contradiction, and the normalized-error identity. The lexer tests establish only preprocessing behavior; they do not invoke a WL evaluator.

The CSV records exact rational samples for four counterfamily choices. For example, for $c = 1/2$, $r = 2$, $n = 1$, targets 16, 32, 64, 128 give errors 64, 256, 1024, 4096 against scale one. These samples illustrate the proved divergence; the proof does not depend on them.

8.2 Candidate source change

To emit a context-bearing diff against an inspected local canonical file:

```
python code/emit_core_guard_patch.py \
/path/to/Asymptotic/src/Kernel/CorePerturbation.wl
```

The script never overwrites that file. The archive also includes the minimal two-line candidate patch. Neither was applied to a real checkout during this audit. `bootstrap_fixed.py` is a candidate replacement helper using the transcribed masking fixture; integrate its logic into the canonical builder rather than treating the archive as an alternate package distribution.

8.3 Unexecuted runtime probes

After loading the chosen package entry into a fresh kernel:

```
Get["/path/to/archive/code/ProbeCoreComponents.wl"];
TestReport["/path/to/archive/code/ReviewCoreComponents.wlt"]
```

The WLT file specifies the *repaired* contract. Run the observation probe first against the baseline when collecting a reproduction. The syntax probe is separate:

```
Get["/path/to/archive/code/ProbeBootstrapSyntax.wl"];
```

It does not load or alter the package. None of these three files was executed here. A lexical grouping scan of supplied WL files is not a substitute for their parsing and evaluation in the target runtimes.

8.4 Contents and build

The archive contains the article's `.tex` and compiled `.pdf`, Python models and tests, candidate changes, WL/WLT probes, scope and novelty records, exact sample data, and a fixture provenance notice. It contains no checksum files, repository mirror, bundled interpreter, or font files.

Rebuild the PDF from `article/`:

```
pdflatex -interaction=nonstopmode -halt-on-error review.tex
pdflatex -interaction=nonstopmode -halt-on-error review.tex
# Repeat if LaTeX requests another cross-reference pass.
```

9 Conclusion

The new mathematical issue is an information-loss bug at a validation boundary. A simplified sum is sufficient to describe the total forward function but insufficient to certify the separate core and perturbation used by a fixed-parameter theorem. The exact counterfamilies show both a gross failure with a linearly moving offset and a subtler failure with a relatively small logarithmic offset. Checking the structured pair addresses the cause without changing valid fixed-data mathematics.

The new distribution issue is a missing grouping family in a source splitter. Its effect is demonstrated by executed Python chunking experiments and explained by the primary language grammar. A delimiter-aware stack is a small, testable improvement, provided its limited parsing scope remains explicit.

These two findings justify concrete changes and focused validation. They do not justify a claim that the whole package has been certified on Wolfram 15 or Mathics3, and they do not inflate the repository's existing backlog by repeating old findings under new examples.

A Inspected source areas and exclusions

The following is a coverage map, not a correctness certificate. “No additional finding promoted” means that a candidate was either addressed by visible guards, belonged to a recorded prior obligation, lacked an adequate witness, or was outside the evidence available here. It does not mean that the module is bug-free.

Area / representative modules	Inspection focus and disposition
AsymptoticAnalysis.wl	Entry assumptions, exact-number comparisons, coefficient reality, sparse jets, local coordinates, and selected forward/inverse paths. Coefficient-reality handling is material to N01. No new generic arithmetic finding promoted.
CorePerturbation.wl	Full component model, automatic inverse, marker derivative formula, public validation, remainder and target domain. N01 retained.
SeriesOperations.wl, SeriesArithmetic.wl	Arithmetic dispatch, powers/logarithms/exponentials, observables, composition, truncation and differentiation. Several apparent composition issues were ruled out by leading-term/precision guards; known review obligations excluded.
SeriesEnvelopeArithmetic.wl	Composite approach recovery, nonnegative envelopes, independent error propagation, integer versus noninteger powers and unary transports. No additional concrete finding promoted.
SourceCoordinates.wl, LogarithmicScales.wl, ReciprocalLogOperations.wl	Source-chart construction, reconstruction, retained carriers and precision transport. No additional finding promoted.
GammaForward.wl, BarnesForward.wl, GammaInverse.wl, GammaInverseOperations.wl, BarnesInverseChecks.wl	Logarithmic products, parameter/branch restrictions, marker coefficients and specialized residual/observable paths. Nearby previously recorded findings excluded.
SpecialFunctionIdentities.wl, SpecialFunctionAdapters.wl, ParameterizedSpecialFunctions.wl	Identity preprocessing, exact special-function families, Frobenius/-parameter handling and domain restrictions. No new witness promoted.
DirichletSpecialFunctions.wl	Zeta integral comparison, Lerch moment expansion and Taylor bound, term/cutoff handling. Defining-sum integration and derivative-extension items already recorded were excluded.
NativeSpecialFunctions.wl, SpecialFunctionRealDomain.wl	Structured native-series import, real projection, error trees, oscillatory carriers and sufficient real-domain contracts. An adjacent complex-phase question was already recorded; not reissued.
FourierCoefficients.wl, FlatSectorOperations.wl	Coefficient operations, finite sectors, graded omitted tails, polynomial observables and derivative contracts. Prior positional-option and tail-collection findings excluded.
InverseCertificates.wl, InverseFunctionFamilies.wl, NativeCompatibility.wl	Selected exact enclosure primitives, branch/observable constructions and native outcome handling. Existing interval-power and retry issues excluded.
MathicsCompatibility.wl, MathicsCalls.wl, MathicsLists.wl, MathicsCoreFunctions.wl	Installation/scoping adapters, call rewriting, collection helpers and core substitutions. No new execution claim.
MathicsAssumptions.wl, MathicsSimplification.wl, MathicsTaylor.wl, MathicsSpecialFunctions.wl, MathicsCalculus.wl, MathicsNumerical.wl	Conservative proofs, Taylor and Stirling support, calculus and numerical helpers. The narrower realness path is recorded as a reachability limitation for N01, not another finding.
validation/build_standalone.py, validation/test_standalone.py	Full builder/helper and focused test source read. Two helpers transcribed for local execution. N02 retained.

Area / representative modules	Inspection focus and disposition
<code>.github/workflows/standalone.yml</code> ; review indexes and status/intake material	Selected workflow/source provenance and novelty screening. No new CI, receipt, documentation or release-policy recommendation promoted.

B Source locators and bibliography

The repository references below point to the reviewed snapshot. For the Mathics parser and Wolfram documentation, the cited pages were consulted on 10 September 2026; no local runtime was inferred from them. Repository source excerpts in the companion fixture are attributed in `licenses/UPSTREAM-NOTICE.md`.

References

- [1] VladimirReshetnikov/Asymptotic, reviewed commit. [Commit metadata and change record](#).
- [2] `src/Kernel/CorePerturbation.wl`. In particular `corePerturbationModel`, `corePerturbationChooseInverse`, `corePerturbationTerm`, and `corePerturbationConstruct`; component checks at lines 126–128, marker and remainder construction at lines 136–196.
- [3] Coefficient canonicalization and `realPolynomialCondition/realPolynomialQ` in the main kernel module.
- [4] Main kernel entry, Mathics loads, assumption capture and exact-input screening.
- [5] `validation/build_standalone.py`. `executable_text`, `mathics_bootstrap`, and `assemble`.
- [6] `validation/test_standalone.py`. Especially the bootstrap tests near the end of the file, including condition/span preservation.
- [7] `src/Kernel/MathicsAssumptions.wl`. `mathicsRealProof` and the structural `Abs` case.
- [8] `src/Kernel/MathicsSpecialFunctions.wl`. Finite Stirling-model adapter and explicit nonexact tail.
- [9] Retained code-review index and retirement policy.
- [10] Maintained code-review status register; wave intake and proposal summaries referenced therein.
- [11] Wave-1 review index.
- [12] Wave-2 review index.
- [13] Wave-3 review index.
- [14] Wave-4 review index.
- [15] Wave-5 review index, overlap table and evidence boundaries.
- [16] Wave-6 review index, overlap table and evidence boundaries.
- [17] Report 43, article source. The executive assessment and novelty table identify F03 as a `PerturbativeInverse` supplied-core-expression validation issue.
- [18] Report 46, scope and principal finding. Its repair concerns `SourceShift` in the exponential-core constructor, not the two component checks examined here.
- [19] Wolfram Research, *Association*, Wolfram Language documentation.
- [20] Wolfram Research, *LeftAssociation*, character documentation, code U+F113.
- [21] Wolfram Research, *RightAssociation*, character documentation, code U+F114.
- [22] Mathics3, `mathics/core/parser/parser.py`. Consulted routines `p_LessBar`, `parse_seq`, and `e_Semicolon`; public master source as retrieved, not a locally installed interpreter.